

Sliding Window

Basics → Master: Full Theory + LeetCode Problem Bank

This guide is built for a struggle-first learner: every pattern is explained with **why it works**, not just code to copy. Use it as a reference map — attempt each problem cold first (20–30 min), then check the theory/approach notes here only if stuck, and revisit with spaced repetition (Day 1 → Day 3 → Day 7 → Day 21).

Section	Covers
1. What & Why	Core idea, brute-force → optimized intuition
2. Two Core Types	Fixed-size window vs Variable-size (grow/shrink)
3. Java Templates	Reusable skeleton code for both types
4. Problem Bank (Tiers 1-6)	30 curated LeetCode problems, basics → master
5. Fully Worked Examples	3 problems solved with full reasoning + code
6. Monotonic Deque Window	Sliding Window Maximum family (advanced)
7. Common Mistakes	Off-by-one, wrong shrink condition, etc.
8. Practice Roadmap	Order to solve + spaced repetition schedule

1. What is Sliding Window & Why It Works

Sliding Window is an optimization over the brute-force pattern of "check every contiguous subarray/substring". Brute force re-examines overlapping elements again and again — that's where the wasted work is. Sliding Window removes that waste by keeping a running window [left, right] and updating it incrementally instead of recomputing from scratch.

The core insight

If you're checking all contiguous subarrays of an array of size n , brute force is $O(n^2)$ or $O(n^3)$ because for every start index you re-scan forward. But notice: subarray [2,3,4] and [2,3,4,5] share almost everything. Instead of recomputing sum/count/frequency for the new subarray, just **add the new incoming element** (right pointer moves) and **remove the outgoing element** (left pointer moves) from your running state. This turns $O(n \cdot k)$ or $O(n^2)$ into $O(n)$ or $O(n \log n)$.

How to recognize a Sliding Window problem

- The problem asks about a **contiguous** subarray or substring (not subsequence — subsequence usually means DP, not sliding window).
- Keywords: "longest", "shortest", "maximum", "minimum", "at most K", "exactly K", "contains all characters", "sum $\geq/\leq$ target".
- You need to track a running aggregate: sum, count, frequency map, distinct-count, max/min.
- Brute force would be $O(n^2)$ by trying every (start, end) pair — sliding window is the $O(n)$ upgrade.

Anti-pattern warning: If the question allows picking non-contiguous elements (subsequence), sliding window does NOT apply directly — that's a DP / greedy problem instead. Confusing these two is the #1 mistake beginners make.

2. The Two Core Types

2.1 Fixed-Size Window

Used when the window size **k** is given directly in the problem (e.g. "subarray of size k", "substring of length k"). The window size never changes — it just slides one step at a time.

Theory

- Build the initial window of size **k** first (indices 0 to **k**-1).
- Then for every step: add `arr[right]`, remove `arr[left]`, move both pointers forward by 1.
- Update your answer (max/min/sum) at each valid window position.
- Time: $O(n)$. Space: $O(1)$ or $O(k)$ if using a frequency map.

2.2 Variable-Size Window (Grow–Shrink)

Used when the window size is **not fixed** — you're looking for the longest/shortest window that satisfies some condition (sum $\leq$ target, at most **K** distinct chars, no repeating chars, etc.).

Theory — the two-pointer invariant

- **Expand** the window by moving *right* forward, always adding the new element to your running state.
- **Check** if the window has broken the constraint (e.g. sum > target, distinct count > **K**).
- **Shrink** the window from the left (move *left* forward, removing elements from state) while the constraint is violated.
- **Record** the answer — after shrinking, the window is valid again, so update max/min length here.
- Time: $O(n)$ — each element is added once and removed at most once, so total pointer moves are $2n$, not n^2 .

The single most important mental model: right pointer explores, left pointer cleans up. Every element enters the window exactly once and leaves at most once → that's *why* this is $O(n)$ instead of $O(n^2)$, even though it looks like a nested loop.

3. Java Templates (memorize the shape, not the code)

3.1 Fixed-size window template

```
// Max sum of any subarray of size k
public int maxSumFixedWindow(int[] arr, int k) {
    int windowSum = 0, maxSum = 0;

    // 1) build first window
    for (int i = 0; i < k; i++) windowSum += arr[i];
    maxSum = windowSum;

    // 2) slide: add new right element, remove old left element
    for (int right = k; right < arr.length; right++) {
        windowSum += arr[right] - arr[right - k];
        maxSum = Math.max(maxSum, windowSum);
    }
    return maxSum;
}
```

3.2 Variable-size window template (the one you'll reuse most)

```
public int variableWindowTemplate(int[] arr, int target) {
    int left = 0, windowState = 0, best = 0; // windowState = sum / count / distinct etc.

    for (int right = 0; right < arr.length; right++) {
        // 1. EXPAND: absorb arr[right] into windowState
        windowState += arr[right];

        // 2. SHRINK: while window breaks the rule, pull left forward
        while (windowState > target) { // condition depends on the problem
            windowState -= arr[left];
            left++;
        }

        // 3. RECORD: window [left..right] is valid here
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Notice the shape is identical across dozens of LeetCode problems — only **what you track** (windowState) and **the shrink condition** change. Once this template is muscle memory, most sliding window problems become "which state do I track?" instead of "how do I even start?".

4. Problem Bank — Tier 1 to Tier 6 (30 problems)

Solve strictly in order. Don't skip to Tier 3 hashmap problems before Tier 2 grow-shrink is automatic — the shrink logic there is the real prerequisite skill.

Tier 1 — Fixed-Size Window Basics

LC#	Problem	Diff.	Key Concept / Theory
643	Maximum Average Subarray I	Easy	Pure fixed-window template; just track running sum, divide at the end.
1456	Max Number of Vowels in Substring of Length K	Med	Same template — windowState counts vowels instead of sum.
1343	Number of Sub-arrays of Size K & Avg $\geq$ Threshold	Med	Fixed window + compare running sum to threshold*k (avoid float division).
2090	K Radius Subarray Averages	Med	Fixed window of size 2k+1 centered at each index; precompute prefix sums for O(1) window sums.
2461	Max Sum of Distinct Subarrays With Length K	Med	Fixed window + frequency map to check all-distinct condition inside the window.

Tier 2 — Variable-Size Window (Grow-Shrink core skill)

LC#	Problem	Diff.	Key Concept / Theory
3	Longest Substring Without Repeating Characters	Med	windowState = Set/last-seen-index map; shrink while duplicate exists. THE canonical variable-window problem.
209	Minimum Size Subarray Sum	Med	windowState = running sum; shrink while sum $\geq$ target, track min length (shrink-to-find-minimum pattern).
1004	Max Consecutive Ones III	Med	windowState = count of zeros; shrink while zero-count > k.
904	Fruit Into Baskets	Med	Disguised "at most 2 distinct" problem — windowState = frequency map, shrink while distinct types > 2.
1493	Longest Subarray of 1's After Deleting One Element	Med	Same as 1004 but window must be non-empty after deletion — off-by-one trap to notice.
424	Longest Repeating Character Replacement	Med	windowState = char-frequency map + track maxFreq; shrink while (windowLen - maxFreq) > k replacements.

Tier 3 — Hashmap / Frequency-Counting Windows

LC#	Problem	Diff.	Key Concept / Theory
438	Find All Anagrams in a String	Med	Fixed window of size = len(pattern); compare frequency maps each slide (or matched-count trick to avoid full map comparison).
567	Permutation in String	Med	Identical theory to 438, just returns boolean instead of indices.

LC#	Problem	Diff.	Key Concept / Theory
76	Minimum Window Substring	Hard	Variable window; shrink while window still satisfies the requirement (opposite direction of Tier 2!) — track 'required vs formed' counts.
340	Longest Substring with At Most K Distinct Characters	Med	windowState = frequency map; shrink while map.size() > k.
159	Longest Substring with At Most Two Distinct Characters	Med	Special case of 340 with k=2 — solve 340 first, this becomes free.
992	Subarrays with K Different Integers	Hard	Key trick: exactly(K) = atMost(K) - atMost(K-1). Combine two variable-window passes.

Tier 4 — Counting Subarrays (the 'exactly K' trick family)

LC#	Problem	Diff.	Key Concept / Theory
1248	Count Number of Nice Subarrays	Med	Odd numbers = 'exactly K' problem → atMost(K) - atMost(K-1), same trick as 992.
1358	Number of Substrings Containing All Three Characters	Med	windowState = last-seen index of a,b,c; once all three present, ALL substrings starting ≤ min(last-seen) and ending at right are valid — add (1+min index) each step.
2444	Count Subarrays With Fixed Bound	Hard	Track last index where element violated min/max bound AND last index of min/max occurrence; window count = right - max(violation, boundary).
1234	Replace the Substring for Balanced String	Med	Invert the problem: find the SHORTEST window outside which all 4 chars already satisfy n/4 count → minimize that window.

Tier 5 — Sliding Window + Monotonic Deque

LC#	Problem	Diff.	Key Concept / Theory
239	Sliding Window Maximum	Hard	Classic monotonic deque: keep indices in decreasing value order, pop from front when out of window, pop from back when smaller than incoming.
1438	Longest Subarray With Abs Diff ≤ Limit	Med	Two monotonic deques (one for max, one for min) inside a variable window; shrink when maxDeque.front - minDeque.front > limit.
862	Shortest Subarray with Sum at Least K	Hard	Prefix sums + monotonic deque (handles negative numbers, so plain two-pointer variable window fails here).
1499	Max Value of Equation	Hard	Sliding window with deque on (y - x) values, window bound by x-coordinate distance, not index distance.

Tier 6 — Master Tier (combined techniques)

LC#	Problem	Diff.	Key Concept / Theory
30	Substring with Concatenation of All Words	Hard	Fixed window of size = words*wordLen, but slide word-by-word (not char-by-char) with a frequency map of whole words.

LC#	Problem	Diff.	Key Concept / Theory
727	Minimum Window Subsequence	Hard	Two-pointer variant needing SUBSEQUENCE match inside window — expand normally, then contract with a backward-matching second pass.
995	Min Number of K Consecutive Bit Flips	Hard	Greedy + sliding window using a difference-array trick to simulate flips in $O(1)$ per step instead of $O(k)$.
220	Contains Duplicate III	Hard	Window of size k + bucket-sort trick (or TreeSet) to check value-difference $\leq t$ in $O(\log k)$ per step.
1425	Constrained Subsequence Sum	Hard	DP where the transition is 'max of last k dp values' — solved efficiently with a monotonic deque sliding window over DP array, not the input array.
1610	Maximum Number of Visible Points	Hard	Convert angles, sort, then run a circular sliding window (duplicate the array to handle wraparound) to find max points within a fixed angle range.

5. Fully Worked Examples (with reasoning)

Example A — 209. Minimum Size Subarray Sum (Tier 2 shrink-to-minimize)

Problem: Given array of positive integers and target, find the length of the shortest contiguous subarray whose sum $\geq$ target.

Reasoning

- Since all numbers are positive, growing the window only increases the sum, and shrinking only decreases it — this monotonic property is exactly what makes sliding window valid here.
- Expand right until sum $\geq$ target, then greedily shrink from left as much as possible while sum stays $\geq$ target — every successful shrink is a candidate for the minimum length.

```
public int minSubArrayLen(int target, int[] nums) {
    int left = 0, sum = 0, minLen = Integer.MAX_VALUE;

    for (int right = 0; right < nums.length; right++) {
        sum += nums[right]; // EXPAND

        while (sum >= target) { // SHRINK while still valid
            minLen = Math.min(minLen, right - left + 1); // RECORD before shrinking further
            sum -= nums[left];
            left++;
        }
    }
    return minLen == Integer.MAX_VALUE ? 0 : minLen;
}
```

Example B — 3. Longest Substring Without Repeating Characters (Tier 2 canonical)

Problem: Find the length of the longest substring without repeating characters.

Reasoning

- windowState = a map from character $\rightarrow$ last index it was seen at.
- As right expands, if the incoming char was seen before AND its last index is inside the current window, jump left directly past that duplicate (no need to shrink one-by-one).
- This is a variant where the shrink is $O(1)$ jump instead of a while-loop — still $O(n)$ overall.

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int left = 0, maxLen = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) {
            left = lastSeen.get(c) + 1; // jump left past the duplicate
        }
        lastSeen.put(c, right);
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

Example C — 76. Minimum Window Substring (Tier 3 master-level hashmap)

Problem: Find the smallest substring of s containing every character of t (with multiplicity).

Reasoning — the 'required vs formed' trick

- Build a frequency map of t → this is what you 'need'.
- Expand right, decrementing need count for each matched char; track 'formed' = how many unique chars have met their required count.
- Once formed == required (unique chars in t), the window is **valid** — now shrink from left while still valid to find the true minimum, updating the answer BEFORE each shrink step.
- This is the opposite of Example A: there you shrunk to escape an invalid state; here you shrink while still **inside** a valid state, trying to make it tighter.

```
public String minWindow(String s, String t) {
    if (s.isEmpty() || t.isEmpty()) return "";

    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    int required = need.size();
    Map<Character, Integer> windowCounts = new HashMap<>();
    int formed = 0, left = 0;
    int[] best = {-1, 0, 0}; // length, left, right

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        windowCounts.merge(c, 1, Integer::sum);
        if (need.containsKey(c) && windowCounts.get(c).intValue() == need.get(c).intValue()) {
            formed++;
        }

        while (formed == required) {
            // valid: try to shrink
            if (best[0] == -1 || right - left + 1 < best[0]) {
                best = new int[]{right - left + 1, left, right};
            }
            char leftChar = s.charAt(left);
            windowCounts.put(leftChar, windowCounts.get(leftChar) - 1);
            if (need.containsKey(leftChar) && windowCounts.get(leftChar) < need.get(leftChar)) {
                formed--;
            }
            left++;
        }
    }
    return best[0] == -1 ? "" : s.substring(best[1], best[2] + 1);
}
```

6. Monotonic Deque Window (239 family theory)

When a variable window needs the **max or min** of the current window efficiently (not just sum/count), a plain variable stays $O(1)$ per step, but recomputing max after every shrink is $O(k)$. The fix is a **monotonic deque** that stores indices, not values.

Invariant to memorize

- The deque holds indices whose values are in decreasing order (for max) from front to back.
- Before adding `nums[right]`: pop from the back while `nums[back] ≤ nums[right]` — those elements can never be the max again since right is bigger and will outlive them.
- Front of deque is always the current window's max index. Pop from the front when it falls outside the window (`front ≤ right - k`).

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> dq = new ArrayDeque<>(); // stores indices
    int[] result = new int[nums.length - k + 1];

    for (int i = 0; i < nums.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst(); // out of window
        while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast(); // maintain decreasing order
        dq.offerLast(i);
        if (i >= k - 1) result[i - k + 1] = nums[dq.peekFirst()];
    }
    return result;
}
```

7. Common Mistakes (from real interview/LeetCode failures)

- **Wrong shrink direction:** shrinking while the window is INVALID (Tier 2 style) vs shrinking while it's VALID to minimize (Tier 3 style, e.g. problem 76) are opposite loop conditions — mixing them up is the #1 bug.
- **Off-by-one on window length:** length is always $right - left + 1$, not $right - left$. Write this on paper before coding.
- **Forgetting to update the answer at the right moment:** for 'minimum window' problems, record the answer *before* or *during* the shrink loop, not only after it exits.
- **Using sliding window on non-contiguous problems:** if the question says "subsequence" (not subarray/substring), sliding window alone won't work — that's DP territory.
- **Negative numbers break monotonic sum window:** Tier 2's simple 'shrink while sum > target' assumes all-positive input. With negatives (e.g. problem 862), you need prefix sums + monotonic deque instead.
- **Frequency map memory leak:** after decrementing a count to 0, remove the key from the map for ``size() == distinct count`` checks (Tier 3) — otherwise distinct-count logic silently breaks.

8. Practice Roadmap & Spaced Repetition

Suggested order (2-3 problems/day)

- **Days 1-3:** Tier 1 (all 5 problems) — get the fixed-window slide motion into muscle memory.
- **Days 4-8:** Tier 2 (all 6 problems) — this is the highest-leverage tier; almost every later tier builds on this shrink-while-invalid skill.
- **Days 9-13:** Tier 3 (all 6 problems) — hashmap state tracking, and problem 76 to contrast shrink-while-valid.
- **Days 14-17:** Tier 4 (all 4 problems) — the `atMost(K)-atMost(K-1)` counting trick shows up repeatedly on LeetCode, worth over-learning.
- **Days 18-21:** Tier 5 (all 4 problems) — monotonic deque; do 239 twice before moving on, everything else in this tier depends on it.
- **Days 22-26:** Tier 6 — attempt cold, 30-40 min each, no looking at solutions before real struggle.

Spaced repetition after first solve

- Day 1: solve with full struggle (aim 25-30 min before hints).
- Day 3: re-solve cold, no notes — this is where the pattern either sticks or reveals a gap.
- Day 7: re-solve cold again, and write down (one line) which sub-decision was hardest (shrink condition? state to track? boundary?).
- Day 21: final cold re-solve — if this one is fast, the pattern is now recall, not recognition.

This mirrors your existing struggle-first / spaced-repetition method — this PDF is meant to be the reference you check *after* attempting, not a shortcut around the attempt.